Security Considerations

This section describes the security-related aspects of the ML-KEM implementation. It is divided into three main subsections:

1. Constant-time leakage
2. Cache-related leakage
3. Timely zeroization of sensitive data

This implementation is designed to minimize:

- timing leakage;
- secret-dependent branching;
- secret-dependent memory access;
- accidental retention of sensitive data in memory.

Testing was performed on:

- x86-64;
- GCC;
- Clang.

This implementation does not guarantee protection against:

- power analysis attacks;
- electromagnetic (EM) side-channel attacks;
- speculative execution attacks;
- all possible microarchitectural side-channel attacks.

For a better understanding of the implementation and additional security validation details, it is recommended to refer to the test directory of this project:

https://github.com/kstzv/ml-kem/tree/main/tests

1. Constant-Time Leakage

For a better understanding of the implementation and additional validation of constant-time behavior, it is recommended to refer to the dedicated dudect test directory:

https://github.com/kstzv/ml-kem/tree/main/tests/test_dudect_ml_kem

In this description:

- the *server* is the entity that generates and stores the keys, performs decapsulation, and destroys the keys when they are no longer needed;
- the *client(s)* are the entities that read the public key published by the server, perform encapsulation to obtain a ciphertext and shared secret, transmit the ciphertext to the server, and destroy the shared secret when it is no longer needed.

The primary lifecycle is as follows:

1. **ml_kem_create_object**() is called. This function uses the **ml_kem_create_keys.c** module, which contains multiple functions involved in secret generation and therefore must operate in constant time with respect to secret-dependent data. **ml_kem_create_object**() is typically called once by the server for a given key pair.
2. The public message (public key) is published. Using this public message, **ml_kem_encaps_core**() is called to generate the shared secret and ciphertext. This process uses the **ml_kem_encaps.c** module, which also contains functions that must operate in constant time with respect to shared secret and ciphertext generation. **ml_kem_encaps_core**() may be called many times by multiple clients using the same public key.
3. The ciphertext is transmitted from the client to the server.
4. **ml_kem_decaps_core**() is called by the server to perform decapsulation. This function internally uses the **ml_kem_encaps.c and ml_kem_decrypt.c modules**. The **ml_kem_decrypt.c** module also contains functions that process secret-dependent data during shared secret recovery from the ciphertext.
5. When the shared secret and ciphertext are no longer needed, **ml_kem_ciphertext_destroy_core**() is called. This function not only releases allocated memory, but also explicitly zeroizes sensitive data before deallocation.
6. When the keys are no longer needed, **ml_kem_destroy_core**() is called. This function not only releases allocated memory, but also explicitly zeroizes secret data before deallocation.

1.1. Overview of ml_kem_create_object()

This function uses the key generation module (**ml_kem_create_keys.c**). The pool allocation module is not considered here, since it only prepares memory allocation and memory layout, and does not directly process secret-dependent data. The **ml_kem_create_keys.c** module performs the following sequence of operations involving secret data:

1. **ml_kem_generation_entropy**() — function responsible for generating the initial seeds. This function is not suitable for dudect testing, since high-quality entropy generation for the initial seeds can never guarantee constant-time behavior. Therefore, the internal operation sequence of this function is described in detail below:

    - Creation of two local arrays:
        - a 33-byte array for the primary seed;
        - a 64-byte array for the derived seeds used for matrix A and for the secret/noise vectors.
    - Invocation of the random generation function to obtain 32 random bytes for the primary seed (constant-time guarantees are not possible at this stage). The last byte remains unused at this point.
    - Writing the current ML-KEM security level into the last byte.
    - Invocation of SHA3-512 in order to derive 64 bytes from the 32-byte seed (plus the appended security-level byte), producing the seeds for matrix A and for the secret/noise vectors.
    - Zeroization of the temporary primary-seed array.
    - Copying the derived seeds into the corresponding fields of the temporary key-generation structure (**struct ml_kem_temp ***).
    - Zeroization of the temporary 64-byte array.

    Based on the sequence above, it can be concluded that the function does not contain secret-dependent branching or secret-dependent memory access patterns.

2. **ml_kem_create_vector_poly**() directly derives the secret and noise vectors from the secret-vector seed. This function was tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected.

3. ml_kem_gen_polynomial_for_cbd(), which is called internally by the function described in point 2, is responsible for constructing full polynomials for the secret vectors from a byte stream. Since this function directly processes secret-dependent data, it was also tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected.
4. ml_kem_create_public_key() and ml_kem_gen_polynomial_for_matrix() operate on the matrix-A seed in order to construct matrix A and the public key. These functions do not process secret-dependent data. The matrix-A seed is public, as is the public key derived from it. Therefore, the author does not consider constant-time leakage testing necessary for these functions.
5. ml_kem_convert_to_bytes_format_and_copy() only performs memory-copy operations on secret-related data using standard C library functionality. The number of copied bytes is not secret-dependent. Therefore, constant-time leakage testing was not performed for this function.
6. ml_kem_finally_create_z_and_h_pk() generates the secret value z. Since this function internally relies on entropy generation, it cannot be meaningfully subjected to constant-time leakage testing. The function operates in the following sequence:
    - Obtaining 32 random bytes for z from the entropy source.
    - Computing the hash of the public key using SHA3-256.

1.2. Overview of ml_kem_encaps_core()

During encapsulation, this function uses the ml_kem_encaps.c module, whose main function, ml_kem_encapsulation(), performs the following sequence of operations:

1. Performs parameter validation and preliminary cleanup of structure fields.
2. Declares local buffers.
3. Depending on the execution context, either:
    - generates a seed for the temporary secret;
    - or copies externally provided data into the internal buffers.
4. Processes the temporary secret data.
5. Computes a SHA3-512 hash of the seed in order to derive:

- the temporary secret;
- and the seed used for ciphertext construction.

The temporary buffer is then zeroized once it is no longer needed.

6. Calls **ml_kem_unpack_pk_t_u16()**, a function responsible for unpacking the public-key message into a format suitable for internal computation. This function does not process secret-dependent data and therefore was not subjected to constant-time leakage testing.
7. Calls **ml_kem_create_temp_secret_y()** in order to derive the temporary secret. This function directly processes secret-dependent data and was therefore tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected.
8. Calls **ml_kem_create_u()**. This function processes secret-dependent data and directly contributes to ciphertext generation. However, it was not tested with dudect because, in addition to secret processing, it also includes noise caused by functions that reconstruct matrix A from the public seed.

   Since the matrix-A generation functions themselves do not process secret-dependent data, the author intentionally chose not to redesign these functions specifically for constant-time behavior in order to avoid unnecessary code complexity.

   From a security perspective, **ml_kem_create_u()** performs the following sequence of operations:

   - creation of local buffers and counters for the u-vector seed;
   - execution of nested loops that reconstruct matrix A polynomial-by-polynomial from the seed;
   - multiplication of the matrix by the temporary secret within the inner loop;
   - conversion into the NTT domain and addition of noise within the outer loop;
   - zeroization of all local buffers and temporary encapsulation fields after completion.

Based on the sequence above, it can be concluded that the function does not contain secret-dependent branching or secret-dependent memory access patterns. This function internally calls routines from the **ml_kem_ntt_main.c** module, which is described in a separate section.

9. Calls ml_kem_create_v() in order to derive the temporary secret contribution for v. This function processes secret-dependent data and was therefore tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected.
10. ml_kem_compress_ciphertext() and its internal static helper functions perform packing of u and v into the final ciphertext message. The computed values u and v are no longer considered secret at this stage. Therefore, analyzing the ciphertext-packing functions for constant-time leakage is not considered meaningful.

The ml_kem_encaps_core() function itself performs only limited processing of secret-dependent data. Its primary responsibilities are:

- preparing memory for the ciphertext;
- allocating the encapsulation structure;
- calling ml_kem_encapsulation();
- receiving the populated encapsulation structure;
- copying the required fields into output buffers;
- zeroizing and destroying the temporary encapsulation structure;
- returning the resulting buffers to the caller.

1.3. Overview of ml_kem_decaps_core()

This function performs the decapsulation process. The initial instructions of this function do not process secret-dependent data and perform only the following operations:

- input validation;
- searching for and atomically acquiring a slot from the decapsulation pool.

The first operation that directly processes secret-dependent data is the call to the main decryption function, ml_kem_decrypt(). The following is a detailed overview of the execution flow of this function:

1. The ciphertext is copied into the appropriate field of the dedicated decryption structure using standard C library functionality.

2. ml_kem_decompress() is called. This function was not tested for constant-time leakage because it does not process secret-dependent data. It reconstructs the computation-ready vectors u and v from the ciphertext, and these vectors are not considered secret by themselves.
3. ml_kem_decrypt_get_poly() is called. This function processes secret-dependent data because it gots the secret vector from the u and v vectors. Therefore, this function was tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected.
4. ml_kem_decrypt_get_seed_m() is called. This function derives the final shared secret from the reconstructed secret vector. In order to perform this operation, ml_kem_decrypt_get_seed_m() internally calls ml_kem_decode1_bit(). Both functions were tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected.

   All secret-dependent data is stored inside the dedicated decryption structure.

Next, ml_kem_encapsulation() is called in order to perform re-encapsulation. Unlike the client-side encapsulation process, the decapsulation flow must explicitly provide the corresponding secret-dependent fields as separate parameters (see the documentation for additional details).

The behavior of ml_kem_encapsulation() was already described in detail in subsection 1.2. After that, ml_kem_decaps_ct_select_ss() is called. This function processes secret-dependent data and therefore must operate in constant time.

The purpose of this function is:

- to compare the ciphertexts;
- and to determine which value should be written into the output buffer:
    - the hash derived from z;
    - or the recovered shared secret.

Therefore, this function was tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected. After ml_kem_decaps_ct_select_ss() completes, the decapsulation slot and its associated decryption and re-encapsulation structures are explicitly zeroized.

1.4. Overview of ml_kem_ciphertext_destroy_core() and ml_kem_destroy_core()

These functions act as destructors. Their purpose is to zeroize sensitive data and release allocated memory.

The most important aspect here is the reliability and effectiveness of the zeroization process, ensuring that secret-dependent data cannot be recovered after memory cleanup and deallocation.

This is necessary in order to reduce the risk of sensitive-data leakage and to prevent incorrect behavior of the implementation caused by residual secret data remaining in memory.

1.5. Overview of the math_operations.c Module

This module performs arithmetic operations in the ML-KEM ring. It is an internal low-level module used by higher-level functions implementing ML-KEM logic. The module provides the following operations:

- modular subtraction through **ml_kem_sub_mod**() and the internal helper function **ml_kem_ct_sub_if_ge**();
- modular addition through **ml_kem_add_mod**();
- modular multiplication through **ml_kem_multipl_mod**();
- modular reduction through **ml_kem_barrett_reduce**().

All of the functions listed above were tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected.

1.6. Overview of the ml_kem_ntt_main.c Module

This module performs polynomial operations in the NTT domain, including multiplication, addition, and conversion between coefficient and point-value representations. The implementation follows the ML-KEM specification defined in FIPS 203. This is an internal low-level module used by higher-level functions implementing ML-KEM logic.

The module provides the following functions:

- ml_kem_ntt_fips203() — converts polynomials into the NTT domain;
- ml_kem_intt_fips203() — converts polynomials back into the coefficient domain;
- ml_kem_multiply() — performs multiplication in the NTT domain;
- ml_kem_add() — performs addition in the NTT domain.

All four functions listed above were tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected.

1.7. Overview of the ml_kem_shake.c and ml_kem_sha3.c Modules

These modules implement the SHA3-256, SHA3-512, SHAKE128, and SHAKE256 operations used throughout the ML-KEM implementation. The modules provide the following wrapper functions:

- ml_kem_shake128();
- ml_kem_shake256();
- ml_kem_sha3_256();
- ml_kem_sha3_512().

All four functions listed above were tested for constant-time leakage using dudect. During testing, no statistically significant signs of timing leakage were detected.

2. Cache-Related Leakage

This implementation does not use:

- secret-dependent array indexing;
- secret-dependent table lookups;
- secret-dependent memory access patterns.

During implementation, the author intentionally avoided constructions in which:

- secret-dependent values;
- bits of secret polynomials;
- shared secrets;

- or temporary secrets

influence:

- array indexing;
- memory-address selection;
- or processor cache access patterns.

Memory-access operations are performed only through:

- fixed indices;
- linear memory access;
- or indices derived exclusively from public algorithm parameters.

The implementation does not use lookup tables indexed by secret-dependent data.

3. Timely Zeroization of Sensitive Data

The userspace implementation contains its own explicit zeroization function, **ml_kem_memzero(),** which uses volatile memory access in order to reduce the risk of the zeroization operation being removed by compiler optimizations. The **ml_kem_memzero()** function is regularly used to clear:

- shared secrets;
- temporary seeds;
- secret vectors;
- temporary polynomial buffers;
- ciphertext buffers;
- internal SHA3/SHAKE states;
- encapsulation and decapsulation contexts.

Zeroization is performed after the corresponding sensitive data is no longer required by the implementation. When porting this implementation to other environments, the author recommends using:

- platform-specific secure zeroization primitives;
- or local explicit memory-wipe functions,

instead of the userspace implementation of **ml_kem_memzero**(). The kernel implementation uses the corresponding kernel-space memory-cleanup mechanisms.

4. Final Notes

If you have additional questions, it is recommended to refer to the documentation for a more detailed understanding of the implementation. If further questions remain, or if you have remarks, criticism, or security-related observations regarding the implementation, you may contact the author using the email address provided in the README of this repository. Constructive criticism is welcome and may help identify weaknesses and improve the security properties of the implementation.